Mobile Demo Script Explanation -- Serial Intent (App Dev, Challenge 09)

Olawoyin Joseph Pamilerin, ISU520196

In one sentence: Java's 'private' keyword is a compile-time-only concept -- reflection calls the method anyway at runtime.

Background -- what the terms below mean

What does "App Dev" mean, as a category? On this platform, **App Dev** challenges ask you to write your own small Android app that behaves a certain way. This is different from **Reversing** or **Exploitation** challenges, where you're given someone else's app to break into -- here there's usually no separate target app to download at all. You are writing code from scratch (or talking to a small server the challenge provides) to meet a spec.

What is an APK? The installable file for an Android app (like a .exe on Windows). For this category, the APK that matters is the one **you build and submit**, not a downloaded target.

What is logcat / the MOBISEC tag? Android apps can write messages to the phone's system log, called **logcat**. The grading harness watches for lines tagged **MOBISEC** and checks their content to decide whether to accept the submission.

How the APK was obtained

This one doesn't hand you a separate target APK to download at all. The challenge is to **write your own small app** that satisfies a spec (often by talking to a small server the platform runs, or to another running app component). So the only APK in play here is the one **you** build and submit -- there's nothing to decompile.

How the solution was built

Built as a normal **Android Studio project** (Gradle). From a terminal: `cd` into its `Android Studio Project` folder and run `gradlew.bat assembleDebug`. The resulting APK is kept right there as **SUBMITTED APK Challenge 09 Serial Intent.apk**, ready to re-upload.

The vulnerability / technique, in plain terms

App-dev task: receive a Serializable `FlagContainer` from the target's exported activity and call its private `getFlag()` method via reflection (private is a compile-time concept only, not a runtime security boundary).

Files to open

1. MainActivity.java:

```
C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\App Dev/09 SerialIntent\Android Studio Project\app\src\m
```

This is our own code -- open directly in Android Studio or any text editor.

What to say -- explanation of the exploit, step by step

1. I found the target's exported activity sends back a Serializable FlagContainer object.

2. I received it and used reflection to force its private getFlag() method callable, since Java's "private" is a compile-time restriction only.

3. I built and submitted the APK; the platform confirmed the flag was extracted successfully.

If an emulator isn't available

If asked how you got/confirmed the flag: "I built my exploit app and uploaded the APK to the platform's own submission page. Their server ran it automatically against the target and returned the flag in the analysis log." This is backed by a real screenshot of that exact submission page (see **Real Screenshot: Live Platform Submission Page** in this challenge's Report.pdf), not just a claim -- worth pointing to directly if an emulator isn't running.

Result

The flag this challenge produced:

```
MOBISEC{HOW_DID_YOU_DO_IT_THAT_WAS_SERIALLY_PRIVATE_STUFF1!!1!eleven!}
```

Source: Verbatim from /apk/81675 analysis log

The real, verbatim output this run produced:

```
shuffling
sending back intent
FLAG: MOBISEC{HOW_DID_YOU_DO_IT_THAT_WAS_SERIALLY_PRIVATE_STUFF1!!1!eleven!}
```